GRAPH DSA — ADVANCED ADD-ON

Java Interview Supplement | Missing Implementations & Advanced Patterns

Use this with the Complete Graph DSA Java Interview Master Notes. This supplement fills implementation gaps and adds advanced graph patterns frequently seen in demanding software engineering interviews.

1. Dijkstra — Complete Java Implementation

Use for single-source shortest paths when every edge weight is non-negative. The priority queue stores {vertex, distance}; stale entries are skipped. Use long for distances.

```
import java.util.*;

class DijkstraFull {
    static class Edge {
        int to, weight;
        Edge(int to, int weight) {
            this.to = to; this.weight = weight;
        }
    }

    static long[] shortestPath(List<List<Edge>> g, int src) {
        int n = g.size();
        long INF = Long.MAX_VALUE / 4;
        long[] dist = new long[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;

        PriorityQueue<long[]> pq =
            new PriorityQueue<>(Comparator.comparingLong(a -> a[1]));
        pq.offer(new long[]{src, 0});

        while (!pq.isEmpty()) {
            long[] cur = pq.poll();
            int u = (int) cur[0];
            long d = cur[1];

            if (d != dist[u]) continue; // stale entry

            for (Edge e : g.get(u)) {
                if (e.weight < 0)
                    throw new IllegalArgumentException("Negative edge");
                long nd = d + e.weight;
                if (nd < dist[e.to]) {
                    dist[e.to] = nd;
                    pq.offer(new long[]{e.to, nd});
                }
            }
        }
        return dist;
    }
}
```

Complexity: O((V+E) log V) with adjacency lists and a binary heap; O(V+E) storage including graph. For path reconstruction, also store parent[v] = u whenever dist[v] improves.

2. Path Reconstruction with Dijkstra

```
static List<Integer> buildPath(int target, int source, int[] parent) {
    List<Integer> path = new ArrayList<>();
    for (int v = target; v != -1; v = parent[v]) {
        path.add(v);
        if (v == source) break;
    }
    Collections.reverse(path);
    if (path.isEmpty() || path.get(0) != source)
        return Collections.emptyList(); // unreachable
    return path;
}
// On every successful relaxation: parent[e.to] = u;
```

Interview detail: distance and actual route are different outputs. A parent array is required to print the route, not just its length.

3. DAG Shortest Path (Negative Edges Allowed)

A directed acyclic graph can contain negative weights because it has no cycles. Topologically sort, then relax each outgoing edge once.

```
static long[] dagShortest(List<List<Edge>> g, int src, List<Integer> topo) {
    int n = g.size();
    long INF = Long.MAX_VALUE / 4;
    long[] d = new long[n];
    Arrays.fill(d, INF);
    d[src] = 0;

    for (int u : topo) {
        if (d[u] == INF) continue;
        for (Edge e : g.get(u)) {
            d[e.to] = Math.min(d[e.to], d[u] + e.weight);
        }
    }
    return d;
```

```
}
```

Complexity: O(V+E) after topological order; O(V) distance storage. For longest path in a DAG, replace min with max and initialize unreachable states appropriately.

4. 0–1 BFS

For graphs whose edge weights are only 0 or 1. A deque acts like a specialised priority queue: zero-cost relaxations go to the front; one-cost relaxations go to the back.

```
static int[] zeroOneBfs(List<List<Edge>> g, int src) {
    int n = g.size(), INF = Integer.MAX_VALUE / 4;
    int[] dist = new int[n];
    Arrays.fill(dist, INF);
    Deque<Integer> dq = new ArrayDeque<>();
    dist[src] = 0;
    dq.addFirst(src);

    while (!dq.isEmpty()) {
        int u = dq.pollFirst();
        for (Edge e : g.get(u)) {
            int nd = dist[u] + e.weight;
            if (nd < dist[e.to]) {
                dist[e.to] = nd;
                if (e.weight == 0) dq.addFirst(e.to);
                else if (e.weight == 1) dq.addLast(e.to);
                else throw new IllegalArgumentException("Weight must be 0 or 1");
            }
        }
    }
    return dist;
}
```

Complexity: O(V+E). This is not valid for arbitrary non-negative weights.

5. Kosaraju — Complete SCC Implementation

Strongly Connected Components in a directed graph. First DFS records finishing order; transpose the graph; process vertices in reverse finishing order.

```
static void dfsOrder(int u, List<List<Integer>> g,
                     boolean[] vis, Deque<Integer> order) {
    vis[u] = true;
    for (int v : g.get(u))
        if (!vis[v]) dfsOrder(v, g, vis, order);
    order.push(u);
}

static void dfsCollect(int u, List<List<Integer>> rev,
                       boolean[] vis, List<Integer> component) {
    vis[u] = true;
    component.add(u);
    for (int v : rev.get(u))
        if (!vis[v]) dfsCollect(v, rev, vis, component);
}

static List<List<Integer>> kosaraju(List<List<Integer>> g) {
    int n = g.size();
    List<List<Integer>> rev = new ArrayList<>();
    for (int i=0; i<n; i++) rev.add(new ArrayList<>());
    for (int u=0; u<n; u++)
        for (int v : g.get(u)) rev.get(v).add(u);

    boolean[] vis = new boolean[n];
    Deque<Integer> order = new ArrayDeque<>();
    for (int i=0; i<n; i++)
        if (!vis[i]) dfsOrder(i, g, vis, order);

    Arrays.fill(vis, false);
    List<List<Integer>> result = new ArrayList<>();
    while (!order.isEmpty()) {
        int u = order.pop();
        if (!vis[u]) {
            List<Integer> comp = new ArrayList<>();
            dfsCollect(u, rev, vis, comp);
            result.add(comp);
        }
    }
    return result;
}
```

Complexity: O(V+E) time and O(V+E) space. Recursion depth can be O(V); iterative DFS is safer for very deep graphs.

6. Bridges — Complete Tarjan DFS

For an undirected graph, edge (u,v) is a bridge when low[v] > tin[u]. Use edge IDs rather than only parent vertex so parallel edges are handled correctly.

```
import java.util.*;

class Bridges {
    static class Edge { int to, id;
        Edge(int to, int id) { this.to=to; this.id=id; }
    }
    static int timer;
    static List<int[]> answer;

    static void dfs(int u, int parentEdge,
                    List<List<Edge>> g, int[] tin, int[] low) {
        tin[u] = low[u] = timer++;
        for (Edge e : g.get(u)) {
            if (e.id == parentEdge) continue;
            int v = e.to;
            if (tin[v] != -1) {
                low[u] = Math.min(low[u], tin[v]);
            } else {
                dfs(v, e.id, g, tin, low);
                low[u] = Math.min(low[u], low[v]);
                if (low[v] > tin[u])
                    answer.add(new int[]{u, v});
            }
        }
    }

    static List<int[]> findBridges(List<List<Edge>> g) {
        int n=g.size();
        int[] tin=new int[n], low=new int[n];
        Arrays.fill(tin, -1);
        timer=0; answer=new ArrayList<>();
        for (int i=0;i<n;i++)
            if (tin[i] == -1) dfs(i, -1, g, tin, low);
        return answer;
    }
}
```

Build each undirected edge with one unique ID and add two adjacency entries carrying that same ID. Complexity O(V+E).

7. Articulation Points — Tarjan DFS

```
static void articulationDfs(int u, int parentEdge,
        List<List<Edge>> g, int[] tin, int[] low,
        boolean[] isAP, int[] timer) {
    tin[u] = low[u] = timer[0]++;
    int children = 0;

    for (Edge e : g.get(u)) {
        if (e.id == parentEdge) continue;
        int v = e.to;

        if (tin[v] != -1) {
            low[u] = Math.min(low[u], tin[v]);
        } else {
            articulationDfs(v, e.id, g, tin, low, isAP, timer);
            low[u] = Math.min(low[u], low[v]);
            if (parentEdge != -1 && low[v] >= tin[u])
                isAP[u] = true;
            children++;
        }
    }
    if (parentEdge == -1 && children > 1)
        isAP[u] = true;
}
```

Run from every unvisited vertex. Root rule is special: DFS root is an articulation point only if it has more than one DFS-tree child. Complexity O(V+E). The Edge class and edge-ID graph construction are the same as in the Bridges section.

8. Tarjan SCC — Core Implementation

```
static int timer;
static int[] disc, low;
static boolean[] inStack;
static Deque<Integer> stack;
static List<List<Integer>> sccs;

static void tarjanDfs(int u, List<List<Integer>> g) {
    disc[u] = low[u] = timer++;
    stack.push(u);
    inStack[u] = true;

    for (int v : g.get(u)) {
        if (disc[v] == -1) {
            tarjanDfs(v, g);
            low[u] = Math.min(low[u], low[v]);
```

```
        } else if (inStack[v]) {
            low[u] = Math.min(low[u], disc[v]);
        }
    }

    if (low[u] == disc[u]) {
        List<Integer> component = new ArrayList<>();
        int x;
        do {
            x = stack.pop();
            inStack[x] = false;
            component.add(x);
        } while (x != u);
        sccs.add(component);
    }
}

static List<List<Integer>> tarjanSCC(List<List<Integer>> g) {
    int n = g.size();
    timer = 0;
    disc = new int[n]; low = new int[n];
    Arrays.fill(disc, -1);
    inStack = new boolean[n];
    stack = new ArrayDeque<>();
    sccs = new ArrayList<>();
    for (int i=0; i<n; i++)
        if (disc[i] == -1) tarjanDfs(i, g);
    return sccs;
}
```

Complexity O(V+E). Critical distinction: for an edge to a vertex still on the stack, update with disc[v], not low[v].

9. Clone Graph

Interview pattern: use a HashMap from original node to cloned node. BFS or DFS prevents duplicate copies and preserves cycles.

```
class Node {
    int val;
    List<Node> neighbors;
    Node(int v) { val=v; neighbors=new ArrayList<>(); }
}
static Node cloneGraph(Node start) {
    if (start == null) return null;
    Map<Node, Node> map = new HashMap<>();
    Queue<Node> q = new ArrayDeque<>();
    map.put(start, new Node(start.val));
    q.offer(start);

    while (!q.isEmpty()) {
        Node u = q.poll();
        for (Node v : u.neighbors) {
            if (!map.containsKey(v)) {
                map.put(v, new Node(v.val));
                q.offer(v);
            }
            map.get(u).neighbors.add(map.get(v));
        }
    }
    return map.get(start);
}
```

Complexity: O(V+E) time and O(V) map/queue space.

10. Bipartite Matching — Kuhn's DFS Algorithm

For a bipartite graph with left-side vertices and right-side vertices, find a maximum matching. Kuhn's algorithm repeatedly tries to assign each left vertex to a right vertex, rerouting an existing match if possible.

```
static boolean tryMatch(int u, List<List<Integer>> g,
                        boolean[] seen, int[] matchR) {
    for (int v : g.get(u)) {
        if (seen[v]) continue;
        seen[v] = true;
        if (matchR[v] == -1 ||
            tryMatch(matchR[v], g, seen, matchR)) {
            matchR[v] = u;
            return true;
        }
    }
    return false;
}

static int maxMatching(int leftN, int rightN,
                       List<List<Integer>> g) {
    int[] matchR = new int[rightN];
    Arrays.fill(matchR, -1);
    int result = 0;
    for (int u=0; u<leftN; u++) {
```

```
        boolean[] seen = new boolean[rightN];
        if (tryMatch(u, g, seen, matchR)) result++;
    }
    return result;
}
```

Complexity: O(V_left × E) for this basic Kuhn implementation. For larger constraints, learn Hopcroft–Karp: O(E√V). Applications: assigning workers to jobs, students to projects.

11. Max Flow — Edmonds–Karp

Flow network: directed edges with capacities. Repeatedly find an augmenting path using BFS in the residual graph; augment by the path bottleneck.

```
static int edmondsKarp(int[][] cap, int source, int sink) {
    int n = cap.length, flow = 0;
    int[][] residual = new int[n][n];
    for (int i=0;i<n;i++) residual[i] = cap[i].clone();

    while (true) {
        int[] parent = new int[n];
        Arrays.fill(parent, -1);
        Queue<Integer> q = new ArrayDeque<>();
        q.offer(source); parent[source] = source;

        while (!q.isEmpty() && parent[sink] == -1) {
            int u=q.poll();
            for (int v=0;v<n;v++) {
                if (parent[v] == -1 && residual[u][v] > 0) {
                    parent[v]=u; q.offer(v);
                }
            }
        }
        if (parent[sink] == -1) break;

        int add = Integer.MAX_VALUE;
        for (int v=sink; v!=source; v=parent[v])
            add = Math.min(add, residual[parent[v]][v]);

        for (int v=sink; v!=source; v=parent[v]) {
            int u=parent[v];
            residual[u][v] -= add;
            residual[v][u] += add;
        }
        flow += add;
    }
    return flow;
}
```

Complexity: O(VE²) for Edmonds–Karp. This matrix version assumes capacities are aggregated per ordered vertex pair; adjacency-list residual edges are preferred when parallel edges matter. Use long capacities if constraints can exceed int.

12. Graph Isomorphism — Interview Theory

Two graphs are isomorphic if there is a one-to-one mapping between their vertices that preserves adjacency. A quick rejection test: vertex count, edge count, degree multiset, component count, and cycle properties must match. Matching these properties does not prove isomorphism. General graph isomorphism is not normally solved with a simple BFS/DFS trick; practical approaches use canonical labelling/refinement and backtracking.

13. Eulerian vs Hamiltonian — Exact Difference

Concept	Eulerian	Hamiltonian
What must be used/visited?	Every edge exactly once	Every vertex exactly once
Circuit	Return to start after using all edges	Return to start after visiting all vertices
Simple test?	Undirected degree/connectivity criteria	No general simple degree criterion
Standard algorithm	Hierholzer	Backtracking / bitmask DP for small n
Complexity	O(V+E)	Exponential in general

14. Corrections / Clarifications to the Main Graph Notes

• Prim's heap implementation shown in the main notes starts at vertex 0; it correctly throws for disconnected graphs. For a minimum spanning forest, restart from each unvisited vertex.

• The DFS articulation/bridge low-link rules must use edge IDs if parallel edges are allowed; parent-vertex-only logic is insufficient for multigraphs.

• Directed Eulerian conditions require the correct in/out-degree balance plus connectivity among nonzero-degree vertices (often checked via SCC/underlying connectivity depending on path vs circuit). Do not use only the degree test.

• Topological sorting is defined only for directed acyclic graphs. If Kahn processes fewer than V vertices, a directed cycle exists.

• Shortest path and MST are different objectives. Dijkstra optimises each source-to-vertex distance; MST minimises the sum of selected tree edges.

15. Final Interview Checklist

■ Can you choose the right representation from constraints?

■ Can you code BFS/DFS and handle disconnected components?

■ Can you detect directed and undirected cycles correctly?

■ Can you explain why BFS gives shortest paths only for unweighted/equal-weight edges?

■ Can you select BFS, 0–1 BFS, Dijkstra, Bellman–Ford, DAG relaxation, or Floyd–Warshall correctly?

■ Can you implement DSU, Kruskal, and Prim without confusing MST with shortest path?

■ Can you explain tin/low and implement bridges, articulation points, and SCC?

■ Can you distinguish Eulerian edges from Hamiltonian vertices?

■ Can you explain residual edges and augmenting paths for max flow?

■ Can you state time and space complexity and discuss overflow, recursion depth, disconnected graphs, and parallel edges?